Interfaces or Bodies? A 24-Instance Pilot Measurement of ε-Sufficient Context for Language-Model Code Editing in Go

Jude Castiel Johansen
Shockware Limited, independent researcher
castiel@shockware.co.uk ORCID 0009-0002-5644-3073

11 October 2026 *Pilot report, version 1.0*

Abstract

A recent note defines a context as *ε-sufficient* for a code-editing language model when the model's pass rate on it is within ε of its pass rate on the best context in a stated family, and proves that under a bounded reader this property is not closed under superset. This report gives the first measurements. From 282 trunk commits of the GitHub CLI (June–October 2026) screened by fixed, logged rules, 24 fail-to-pass instances were obtained. Holding localisation at the gold write-set, two models edited each instance under nine contexts: task text only; the write-set alone; the write-set plus the exported interfaces of its direct dependencies (A1); the same with full bodies (A1b); A1 plus the interfaces of dependent packages (A1F); and A1 padded with unrelated same-repository code to 32k, 64k, 128k and 200k prompt tokens. Patches were verified by hidden fail-to-pass tests and by the tests of the write-set and its dependents. On Claude Haiku 5.5 (10 samples per cell) interfaces outperformed bodies by +0.04 [+0.01, +0.08] at one third of the tokens, outperformed the write-set alone by +0.07 [+0.02, +0.13], and adding dependent-package interfaces did not help (−0.05). Padding to 200k tokens changed the pass rate by +0.03 [−0.09, +0.15] on Haiku and −0.04 [−0.33, +0.22] on Claude Sonnet 5.5 (3 samples per cell): the degradation curve is flat within the resolution of this pilot. Sonnet solved 14 of 24 instances at least once from the task text alone, Haiku 4 of 24, which makes a task-only control essential for any confirmatory study. Run-to-run variation at default sampling was substantial (25% of Haiku cells non-unanimous). The whole measurement cost about $34 in API calls. Instances, contexts, harness and all 2,273 verdicts are released. This is an exploratory pilot on one repository; it fixes the design of a preregistered study and makes no confirmatory claim.

1 Purpose and relation to prior work

Systems that edit code with a language model choose how much of the repository to show it. The definitions note [4] frames the choice as a question of sufficiency relative to a bounded reader: a context K is ε-sufficient for editor A and contract C when $p_A(K, C) \geq p^*_{A,\mathcal{K}}(C) - \varepsilon$, where p is the probability that the editor's patch passes an oracle of tests and p^* is the best pass rate over a candidate family $\mathcal{K}$. Under a factoring of p into coverage of needed facts times a non-increasing function g of load, the note proves that sufficiency is not closed under superset. That result is a possibility theorem about a stylised reader; whether g actually falls over practical loads, and whether the interfaces of dependencies are as good as their bodies, are empirical questions. The public SuffBench specification [3] fixes an instance format for asking them.

This pilot measures both questions on a small scale, with the purpose of fixing the margin, the controls and the sampling of a preregistered study. Related measurements exist and the design follows them where it can: Sam-Bodden [6] varied the representation of surrounding Python code at fixed localisation and found signatures about as good as source; Mohammadi et al. [5] found availability of needed facts, not position, decides success; Du et al. [2] and Xue [7] report accuracy falling with length even when needed facts are present; hierarchical context pruning [8] found pruning dependent-file bodies harmless for completion; Bjarnason et al. [1] quantify run-to-run randomness in agent evaluation. What is added here is small but specific: a statically typed language with compiler-derived interfaces, verdict-based outcomes with a frame check, padding at verified fixed coverage up to 200k tokens, and a task-only control that turns out to matter.

Arm	Content	Median prompt tokens
A∅	task text only	554
W	write-set bodies	13,476
A1	W + `go doc -all` of direct in-module deps	40,840
A1b	W + bodies of direct deps	119,753
A1F	A1 + `go doc -all` and test names of dependents	51,783
P32–P200	A1 padded to 32k / 64k / 128k / 198k	as named

Table 1: Arms. System prompt and task are included in the token counts.

2 Instances

Mining. First-parent commits of `cli/cli` `trunk` from 1 June to 10 October 2026 were examined in order (282 commits). A commit C with first parent P became a candidate if it changed at least one non-test Go file and at least one test file, touched neither `go.mod` nor `vendor/`, deleted or renamed no source file, changed at most two non-test packages, and changed at most 120 non-test lines. For each candidate the parent was checked out, the child's versions of all changed test files were written over it, and the changed test packages were run: the package had to compile and at least one test had to fail. At the child, the same packages and then the write-set together with every package that depends on it had to be green (13 packages that fail on the Windows baseline were excluded throughout). Finally, tokens introduced by the gold patch into the write-set's sources (identifiers and string-literal words absent from the parent's write-set files) were intersected with tokens in the test diff's added lines; a non-empty intersection rejected the commit. Of 282 commits, 25 passed; the most common rejections were "no source-and-test change" and "touches go.mod". Every rejection is logged with its reason in `instances/rejected.jsonl`.

Veto. One instance was removed on manual inspection: its hidden test asserts the exact list of GraphQL fields the command requests, a requirement no task text states and no context arm expresses. The syntactic leak rule misses it because the field name already occurs in the package. The protocol anticipates such cases (SWE-bench-Verified-style under-specification screening); the veto and its reason are recorded. Twenty-four instances remain: 12 from August–October and 12 from June–July 2026, in 20 distinct packages. Median gold patch: 4 changed non-test lines (range 1–40).

Task text. For 13 instances the linked GitHub issue body is the task. For 11 without a usable issue, a behaviour-only specification was written by Claude Haiku 5.5 from the pull-request text and the test diff, under an instruction to name no identifier, file or implementation, with an automatic check against the gold patch's introduced tokens. The source of each task text is recorded and the author reviewed the rewritten ones before release.

Write-set and slices. The write-set W is the set of packages whose non-test files the gold patch changes (one package in 22 instances, two in 2). Backward and forward slices come from `go list`. The forward slice ranges from 5 to 255 packages (median 6).

3 Design

Arms. Localisation is fixed at W in every arm. Contexts are serialised deterministically with W last, immediately before the task. Table 1 lists the arms with the median prompt size measured by the vendor's token counter for Haiku 5.5. Padding packages are same-module packages outside W, its backward slice and its forward slice, not sharing a parent directory with any W package, taken in an order fixed by a hash of the instance identifier; the pad is placed before A1 and cut at a line boundary to land within 2% of the target. A level is skipped when A1 alone exceeds it (P32 was built for 11 of 24 instances).

Editor protocol. Single shot: system prompt, context and task in; search-and-replace blocks out, applied with an exact matcher. One format-repair turn is allowed only when a block fails to apply; no test feedback is ever given. Sampling parameters are not settable on the models used, so every sample is an independent call at vendor defaults; identical prompts share a prompt cache. Models: `claude-haiku-5-5` on all nine arms with $k = 10$ samples per cell, and `claude-sonnet-5-5` on A∅, A1, P64, P128 and P200 with $k = 3$. Documented training cut-offs were not verified against the commit dates; the task-only arm is the operational control.

Verification. A patch is *applied* if every block matches; *builds* if `go build ./...` succeeds; *Δ-passes* if every fail-to-pass test passes; *frame-passes* if, in addition, every test of W and of every package depending on W passes (scoping the frame check to dependents follows the note's observation that a frame violation can only surface there). Identical patches reuse their verdict. Requests above 900k tokens would

Arm	Tokens	Δ-pass [95%]	Δ-pass ∧ frame	build err.	frame fail.
A∅	554	0.096 [0.012, 0.204]	0.096	1	0
W	13,476	0.579 [0.404, 0.754]	0.525	11	13
A1	40,840	**0.646** [0.467, 0.817]	**0.604**	2	10
A1b	119,753	0.604 [0.433, 0.775]	0.550	4	13
A1F	51,783	0.600 [0.425, 0.771]	0.567	3	8

Table 2: Base arms, Claude Haiku 5.5, 24 instances, 10 samples per cell. Counts are over 240 samples per arm.

	Haiku 5.5 ($k = 10$)		Sonnet 5.5 ($k = 3$)	
Load	n	paired change vs A1 [95%]	n	paired change vs A1 [95%]
P32	10	+0.030 [−0.030, +0.120]	–	–
P64	14	+0.029 [−0.014, +0.093]	9	−0.111 [−0.333, 0.000]
P128	19	−0.011 [−0.126, +0.105]	9	−0.074 [−0.222, 0.000]
P200	18	+0.033 [−0.094, +0.150]	9	−0.037 [−0.333, +0.222]
Including task-only-solvable instances: Haiku +0.03, +0.02, +0.02, +0.03 (n=11, 18, 23, 22); Sonnet –, −0.06, −0.03, −0.02 (n=18, 23, 22).				

Table 3: Change in Δ-pass when A1 is padded to the stated load, excluding instances the model solved from task text alone. n is the number of instances with that arm built.

be recorded as infrastructure errors; none occurred.

Analysis. The unit is the instance. Per-cell pass rates are averaged over instances; intervals are percentile bootstraps over instances (2,000 resamples); paired differences are bootstrapped on per-instance differences. Instances that the model solved at least once from task text alone are excluded from that model's curve, and results are reported both ways. Nothing here is a confirmatory test.

4 Results

Task-only control. Haiku solved 4 of 24 instances at least once with no repository context (pooled pass 0.10); Sonnet solved 14 of 24 (0.44). The solved sets are spread evenly across the two date windows. Whether this is memorisation or issue texts that make the fix obvious to a stronger model cannot be separated here; either way, a confirmatory study must screen instances with a task-only pass on its strongest model.

Interfaces versus bodies (Haiku, 24 instances, Table 2). The write-set plus dependency interfaces (A1) was the best base context: +0.042 [+0.012, +0.075] over the same packages with bodies (A1b) at one third of the tokens, with the per-instance majority outcome never disagreeing between the two ($\pi_d = 0/24$); +0.067 [+0.021, +0.133] over the write-set alone, which produced the most build errors (11) and frame failures (13); and +0.046 [+0.012, +0.083] over A1F, so adding the interfaces of dependents did not reduce frame failures (8 versus 10) and slightly hurt. Nine instances were ceilings on which the write-set alone passed every sample.

Degradation at fixed coverage (Table 3). Padding A1 with unrelated code changed Haiku's pass rate by at most ±0.03 at every level up to 200k tokens, with every interval covering zero. Sonnet's point estimates were negative at every level on its 10 non-trivial instances (−0.11, −0.07, −0.04 at 64k, 128k, 200k), again with intervals covering zero and only three samples per cell. At $\varepsilon = 0.05$ the smallest load, A1, was already within ε of the best load for 22 of 24 instances on both models. Within this pilot's resolution the degradation curve g is flat from 40k to 200k tokens for these single-package Go fixes.

Run-to-run variation. With default sampling, 25% of Haiku cells (48 of 194 with at least three samples) and 11% of Sonnet cells (12 of 111) were non-unanimous, with the minority outcome holding about 30% of the samples in those cells. Per-sample noise is therefore large relative to the arm effects above; the intervals already reflect it.

Cost. The Haiku arm of the pilot consumed 1.44M uncached input tokens, 145M cached, 15M cache-written and 2.4M output tokens, about $4.71 at list prices; the Sonnet arm 0.23M, 18.9M, 9.5M and 0.31M, about $29.16. Tokens per instance scale with the arm: A1 is a third of A1b and a fifth of P200.

5 What this changes in the planned study

1. **Screening.** A task-only pass on the strongest evaluated model becomes a mandatory screening step; instances solved there are excluded or reported separately.

2. **Margin.** The planned rule "ε at least twice the flip rate" is unusable with cell non-unanimity of 0.25; the margin must be set from per-instance pass-rate variance, and $k \geq 10$ per cell is needed on the model that carries the confirmatory contrast. On Haiku-class pricing that is affordable; on frontier pricing it is not, which argues for a cheap dense model plus a sparse frontier check.

3. **Hypotheses.** H2 (interfaces non-inferior to bodies and to the transitive slice) is supported in direction; the pilot estimate is in favour of interfaces. H1 (degradation with load) found no effect up to 200k on this task class; the confirmatory design should either extend load beyond 200k or use tasks with longer dependency chains, and should state that a flat curve is an admissible outcome. H3 (forward slice reduces breakage) found nothing and stays descriptive.

4. **Leak rule.** The syntactic rule must be paired with manual under-specification screening, as the vetoed instance shows.

6 Limitations

One repository and one language; 24 instances, mostly one-package fixes with a median of four changed lines, so the ceiling rate is high and dependency reasoning is shallow. Two models of one vendor, three samples per cell on the larger one, no open-weights model and therefore no temperature-zero measurement. Training cut-offs unverified. Expressibility is syntactic. Eleven task texts were model-written. Padding is of one type (same-repository code with a directory-proximity ceiling); other padding types and the full transitive slice were not run. Pricing is list pricing at the time of the run. Intervals are wide and nothing is corrected for multiplicity; no confirmatory conclusion should be drawn.

7 Availability

Instances (task text, gold patch, hidden tests, write-set, slices, validity record), the mining log with every rejection, the harness, the serialised arm definitions, all 2,273 verdict records and the analysis script are released under the SuffBench specification as `pilot/` in `https://github.com/Shockware-Limited/suffbench`, tag `v0.1.1-pilot`, CC-BY-4.0 for data and text, Apache-2.0 for code. The pilot set is disjoint from any future confirmatory set and will not be reused.

Acknowledgements and AI-use disclosure

The author used Anthropic's Claude Fable 5.1, accessed in October 2026 and referred to by the author as *Lemma*, to design the pilot, to write the mining, context-building, running and analysis code, to run the measurement against the author's API account, and to draft and revise this text; Claude Sonnet 5.5 and Claude Haiku 4.5 were used for parts of the harness work, and Claude Haiku 5.5 wrote the eleven rewritten task specifications under the stated constraints. The author conceived the problem and the benchmark, directed the work, reviewed the task texts, the veto, the code and this report, and takes full responsibility for its content. Generative AI tools are not authors of this work.

References

[1] Bjarni Haukur Bjarnason, André Silva, and Martin Monperrus. On randomness in agentic evals. arXiv:2602.07150, 2026.

[2] Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, et al. Context length alone hurts LLM performance despite perfect retrieval. arXiv:2510.05381, 2025.

[3] Jude Castiel Johansen. SuffBench: Specification for a context-sufficiency benchmark for code editing (v0.1.0-spec). Shockware Limited, `https://github.com/Shockware-Limited/suffbench`, 2026.

[4] Jude Castiel Johansen. ε-sufficient context for bounded readers: Definitions and a non-monotonicity result for code editing. Zenodo, version 1.1, 2026.

[5] Bardia Mohammadi, Lars Klein, Aman Chadha, Akhil Arora, et al. The working set of a coding agent: Coherence debt in repository-scale tasks. arXiv:2608.16630, 2026.

[6] Brian Sam-Bodden. What context does a coding agent actually need to act? arXiv:2607.09691, 2026.

[7] Yue Xue. When and how context rot appears in coding agents: A white-box study of agent skills in code auditing. arXiv:2607.17937, 2026.

[8] Lei Zhang, Yunshui Li, Jiaming Li, Xiaobo Xia, et al. Hierarchical context pruning: Optimizing real-world code completion with repository-level pretrained code LLMs. arXiv:2406.18294, 2024.